

GPAS: OPTIMIZER-AWARE MICRO-BATCH ALLOCATION FOR MULTI-TEACHER ON-POLICY DISTILLATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-teacher on-policy distillation (MOPD) trains one student on its own rollouts, each scored token by token by the specialist teacher for its task. In current methods the task mixture sets both the objective weights and the compute each task receives, so adapting the mixture changes what the student learns and how precisely each task gradient is estimated at the same time. We separate the two. The objective weights are fixed, every task appears in every optimizer step, and the only adaptive quantity is the number of fixed-size micro-batches each task receives. Scaling each micro-batch loss by its task weight divided by its count keeps the expected gradient the same under every allocation, so the allocation changes only the variance of the gradient estimate. Gradient-Preconditioned Adaptive Sampling (GPAS) minimizes this variance in AdamW’s preconditioned coordinates: each task receives micro-batches in proportion to its weight times its gradient standard deviation after AdamW’s diagonal scaling. Cost-GPAS also accounts for the measured time per micro-batch. Both rules use gradients and timings the trainer already produces and add no forward or backward passes. In four-teacher distillation of Qwen3-1.7B on mathematics, code, instruction following, and science, **[TODO: GPAS lowers held-out teacher loss by X% at matched optimizer steps, Cost-GPAS reaches Uniform’s final loss in Y% fewer GPU hours, and no task falls below its Uniform counterpart on downstream benchmarks.]**

1 INTRODUCTION

Multi-teacher on-policy distillation (MOPD) merges several specialist teachers into one student. The student samples a response, the teacher for that task scores every token, and the student is updated to reduce the sampled-token reverse KL to the teacher (Ma et al., 2026). Every MOPD method must choose a task mixture, and the choice matters more than it looks. Open-MOPD reports that, under token-mean aggregation, a $25\times$ difference in response length leaves instruction following with about 1% of the gradient tokens despite 20% of the prompts (Gao et al., 2026). D³-MOPD adapts the mixture online from the teacher-loss gap and its descent rate (Sun et al., 2026). In both cases the mixture is at once the training objective and the compute schedule, so a change in one cannot be separated from a change in the other.

We keep the two apart. The objective is a weighted sum of per-task teacher losses with weights chosen before training. Each optimizer step accumulates G fixed-size micro-batches, each drawn from a single task, and the only online decision is how many micro-batches each task gets. Multiplying each micro-batch loss by its task weight divided by its task’s count makes the expected gradient identical under every allocation (Section 2). The allocation therefore decides only how precisely each task gradient is estimated, and the natural criterion is variance: spend more micro-batches where the gradient is noisier. **[TODO: In our four-teacher run the per-micro-batch gradient noise, measured in AdamW coordinates, differs by up to $X\times$ across tasks and drifts by $Y\times$ over training (Figure 2), so no fixed split is variance-optimal throughout.]**

Gradient-Preconditioned Adaptive Sampling (GPAS) assigns task i a count proportional to $w_i\sqrt{e_i}$, where w_i is its weight and e_i its per-micro-batch gradient noise measured after AdamW’s diagonal scaling. This is the classical stratified-sampling rule, which samples each stratum in proportion to its

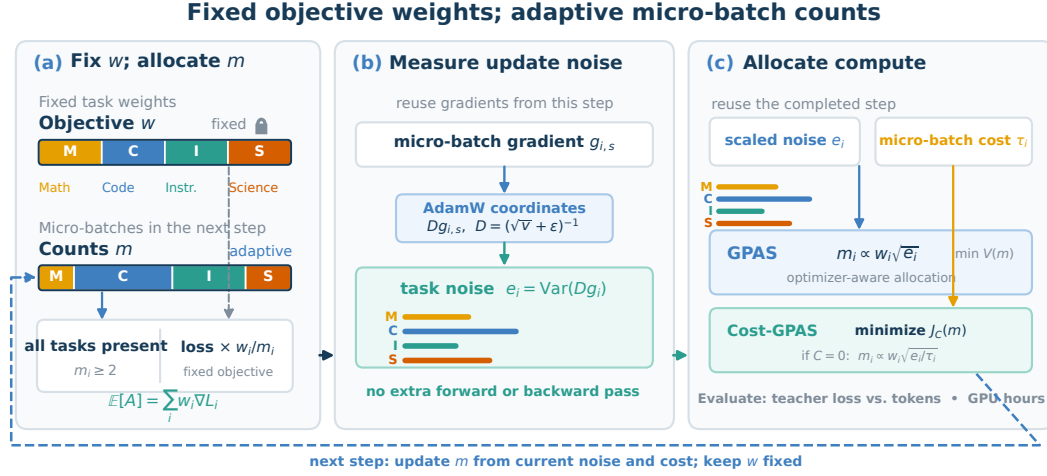


Figure 1: GPAS fixes the objective weights w and adapts only the micro-batch counts m . Gradients from the completed step give each task’s noise in AdamW coordinates, and measured time adds the cost correction, so the next allocation needs no additional model passes.

standard deviation, applied in the coordinates the optimizer actually uses. The coordinates matter: raw and scaled noise can rank tasks differently, and allocating by raw noise can then increase the variance that AdamW sees (Section 3). Cost-GPAS additionally uses the measured time per micro-batch and the fixed time per step, so that variance is minimized per GPU hour instead of per step. Both rules run on statistics from the step just completed, namely each task’s within-step gradient variance and its timing. Every task is present in every step, so these statistics are always available, and computing them needs one parameter-sized buffer and no extra forward or backward pass. GPAS is a drop-in change to a standard MOPD trainer with AdamW.

[TODO: Findings, to be written from the four-teacher runs: (i) how much the scaled noise levels differ across tasks and drift, and in what fraction of steps raw and scaled noise rank the tasks differently; (ii) the held-out teacher-loss reduction of GPAS at matched steps; (iii) the GPU-hour saving of Cost-GPAS at Uniform’s final loss; (iv) per-task downstream results relative to Uniform.]

Our contributions are:

- a fixed-objective MOPD estimator in which adaptive micro-batch counts change the variance of the gradient estimate but not its expectation;
- GPAS and Cost-GPAS, optimizer-aware count rules whose statistics come from the passes MOPD already performs;
- a four-teacher study of Qwen3-1.7B that measures noise and allocation dynamics, held-out loss per step and per GPU hour, and per-task downstream capability, with the recipe and logs released.

2 SETTING: FIXED-WEIGHT MOPD WITH ADAPTIVE COUNTS

A micro-batch contains $b = 4$ prompts from one task, each with one sampled response, and is processed in one backward pass. One optimizer step accumulates $G = 16$ micro-batches, 64 responses in total, before one AdamW update. As in MOPD and Open-MOPD, each prompt receives one response and each rollout batch is used for one update (Ma et al., 2026; Gao et al., 2026), so every micro-batch consists of fresh prompts.

There are M tasks (domains), each with a fixed teacher. Let $L_i(\theta)$ be task i ’s sampled-token reverse-KL loss, averaged over the valid tokens of a response and then over the b responses in a micro-batch

(Appendix A.6). Fixed positive weights w_i summing to one define the objective

$$F(\theta) = \sum_{i=1}^M w_i L_i(\theta). \quad (1)$$

Our experiments set w_i proportional to the inverse of task i 's loss at the initial student; any fixed choice works.

Task i receives an integer count m_i with $\sum_i m_i = G$ and $m_{\min} \leq m_i \leq m_{\max}$; we use $m_{\min} = 2$ and $m_{\max} = 8$. Let $g_{i,s}$ be the gradient of micro-batch s of task i . The accumulated gradient is

$$A = \sum_{i=1}^M \frac{w_i}{m_i} \sum_{s=1}^{m_i} g_{i,s}, \quad (2)$$

which the trainer obtains by multiplying each micro-batch loss by w_i/m_i before its backward pass. Conditional on the student state and the counts, $\mathbb{E}[A] = \nabla F(\theta)$ for every allocation, so the counts change the variance of the gradient estimate but not the objective. Averaging all tokens of a step together would instead give task i an effective weight that grows with both its count and its response length; this is the length-driven imbalance that Open-MOPD reports, and the per-response mean together with the w_i/m_i factor removes it by construction. Table A1 in the appendix lists the notation.

3 GPAS MICRO-BATCH ALLOCATION

3.1 NOISE IN OPTIMIZER COORDINATES

AdamW divides coordinate k of the gradient by $\sqrt{v_k} + \epsilon$, so noise in coordinate k affects the update in proportion to $D_k = (\sqrt{v_k} + \epsilon)^{-1}$. We therefore measure gradient noise in these coordinates, treating D as fixed within a step and micro-batches as independent. Let e_i be the noise of one task- i micro-batch and $V(m)$ the noise of the accumulated gradient A :

$$e_i = \mathbb{E} \|D(g_i - \mathbb{E}[g_i])\|_2^2, \quad V(m) = \mathbb{E} \|D(A - \nabla F)\|_2^2 = \sum_{i=1}^M \frac{w_i^2 e_i}{m_i}. \quad (3)$$

For a fixed student state and step size, the allocation enters the one-step descent bound only through $V(m)$ (Appendix A.2). Minimizing $V(m)$ subject to $\sum_i m_i = G$ gives

$$m_i \propto w_i \sqrt{e_i}. \quad (4)$$

We call this rule Gradient-Preconditioned Adaptive Sampling (GPAS). It is Neyman allocation, the classical rule that samples each stratum in proportion to its standard deviation, applied in the coordinates AdamW uses. The continuous counts are clipped to $[m_{\min}, m_{\max}]$, the unclipped ones rescaled to sum to G , and the result rounded by largest remainder (Appendix A.3).

The coordinates matter because raw and scaled noise can rank tasks differently. In a controlled three-task example whose scaling reverses the ranking, allocating by raw noise raises the scaled variance to $2.3\times$ that of Uniform, whereas GPAS lowers it to $0.68\times$ (Appendix A.5).

3.2 COST-GPAS

Long-response tasks cost more per micro-batch, so the allocation with the least variance per step need not have the least variance per hour. Let τ_i be the measured time of one task- i micro-batch and C the fixed time per step for rollout, synchronization, and the optimizer update, so that a step takes $C + \sum_i m_i \tau_i$. A wall-clock budget T then allows $T/(C + \sum_i m_i \tau_i)$ steps, and the variance of the gradient averaged over those steps is proportional to

$$J_C(m) = \left(C + \sum_{i=1}^M m_i \tau_i \right) V(m). \quad (5)$$

Cost-GPAS minimizes J_C over the feasible integer counts, a small enumeration for four tasks. With $C = 0$ the minimizer is $m_i \propto w_i \sqrt{e_i/\tau_i}$ (Appendix A.3); as C grows relative to $\sum_i m_i \tau_i$, the minimizer approaches GPAS.

3.3 ONLINE ESTIMATES AND IMPLEMENTATION

Every statistic for the next allocation comes from the step just completed. Let $\bar{g}_i$ be the mean of task i 's micro-batch gradients in that step. Then

$$\hat{e}_i = \frac{1}{m_i - 1} \sum_{s=1}^{m_i} \|D(g_{i,s} - \bar{g}_i)\|_2^2 \quad (6)$$

is the unbiased within-step estimate of e_i , and $m_{\min} = 2$ makes it available on every step. The allocation uses the current estimate without smoothing. **[TODO: Confirm from the logged step-to-step change of $\hat{e}_i$; add smoothing only if the trace is unstable.]** Response lengths make timings noisy, so τ_i and C are exponential averages of measured values with decay 0.9. We also log the predicted variance ratio relative to Uniform,

$$H = \frac{V(m^{\text{unif}})}{V(m)}, \quad (7)$$

which is at most 2 because Uniform assigns 4 micro-batches and $m_i \leq 8$. The first step uses Uniform because no statistics exist yet.

Computing $\hat{e}_i$ needs one parameter-sized buffer for $\bar{g}_i$, a few elementwise passes over the parameters per micro-batch, and one scalar all-reduce; it adds no forward or backward pass. GPAS runs with conventional AdamW. The allocation changes the noise term of AdamW's second moment, as any change in batch composition does; Appendix A.4 bounds this change under the count limits and describes an optional allocation-independent second-moment observation that the experiments do not use. Algorithm 1 summarizes one step.

1. Choose bounded integer counts m from the previous step's $\hat{e}_i$ (and τ_i , C for Cost-GPAS); use Uniform on the first step.
2. For every task i , take $m_i b$ new prompts, sample one response per prompt, obtain token-level teacher feedback, and accumulate each micro-batch gradient with weight w_i/m_i .
3. From the same gradients and execution trace, compute $\hat{e}_i$, update the timing averages, and log counts, losses, tokens, H , and timing.
4. Apply one AdamW update.

Algorithm 1: One optimizer step with GPAS.

4 EXPERIMENTAL SETUP

We distill four specialist teachers, for mathematics, code, instruction following, and science, into one Qwen3-1.7B student. Table A2 in the appendix lists the models, data, hyperparameters, and system configuration; entries in red remain to be filled from the final runs.

4.1 DATA AND BUDGET

Each task has a fixed stream of 16,000 prompts, shuffled once and consumed without replacement. All methods read a task's stream in the same order, so the n th prompt a method requests from a task is the same prompt for every method; the allocation determines only how far into each stream a method has progressed at a given step. Training runs for 500 steps of 64 responses, 32,000 responses in total. A task at the maximum count on every step consumes exactly $500 \times 8 \times 4 = 16,000$ prompts, so no prompt is ever reused. The objective weights are the normalized inverse initial task losses, measured once at the initial student and shared by all runs.

4.2 METHODS

Uniform assigns (4, 4, 4, 4) micro-batches on every step. GPAS uses Equation 4 and Cost-GPAS minimizes Equation 5. Two further count rules use the same estimator with a different signal. RawNoise applies the GPAS rule to the unscaled noise $\mathbb{E}\|g_i - \mathbb{E}[g_i]\|_2^2$ instead of e_i , isolating the

effect of the optimizer coordinates. LossGap sets $m_i \propto w_i \bar{L}_i$, where $\bar{L}_i$ is an exponential average of task i 's training-batch teacher loss; this is the signal that gap-following mixtures use, placed inside the fixed-objective estimator. StdMOPD is the common recipe: the same 16 prompts per task per step as Uniform, with the sampled-token loss averaged over all valid tokens of the step, so its effective task weights follow token counts. Its objective differs from Equation 1, and we compare it on per-task held-out losses and downstream scores. All methods share the optimizer, learning-rate schedule, count bounds, response cap, and prompt streams, and every adaptive rule starts with one Uniform step. Each method runs with one seed; the seed sets the prompt shuffle and the response sampling, and streams are matched across methods.

4.3 MEASUREMENTS

Held-out teacher loss. The primary training metric is the weighted teacher loss $\sum_i w_i L_i$ on a fixed held-out set of 128 prompts per task, evaluated from checkpoints every 50 steps with a fixed sampling seed. Differences between methods carry paired bootstrap intervals over the 512 held-out prompts; the intervals reflect evaluation sampling, not seed variation. Training-batch losses are reported in the appendix because they depend on which prompts each method drew. The common loss target for compute comparisons is Uniform's final held-out loss.

Compute. GPU hours are full wall time times the two reserved GPUs, including synchronization and waiting. Each step's trace records rollout time and per-task scoring and backward time, from which τ_i and C are obtained (Appendix A.8).

Held-out gradient variance. At early, middle, and late Uniform checkpoints, we generate 32 fresh micro-batches per task, estimate $\hat{e}_i$ on each half, and evaluate the allocation each rule computes from one half on the noise measured on the other half (Appendix A.7). This tests whether the within-step estimate predicts fresh noise and how much variance each rule removes, independently of the online ratio H .

Capability. We report MATH-500 greedy pass@1, a fixed LiveCodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4 for the initial student, each teacher, and each method, together with the per-task normalized gain $(s - s_{\text{init}})/(s_{\text{teacher}} - s_{\text{init}})$ and its average. **[TODO: Confirm benchmark versions and decoding settings; add AIME24/25 average@16 or a second science benchmark if the initial student leaves too little headroom.]** We also report mean response length and truncation rate per task.

4.4 SYSTEM

Each run reserves two GPUs: one trains the student, and one serves the student's rollouts and holds the four teachers, which score their tasks' responses in a fixed order. Generation stops at 4,096 tokens per response and truncated responses are kept, because the token-level loss needs no completed response. The cap bounds the rollout time of a step, which does not depend on the counts, so the step time is predictable from the counts through $C + \sum_i m_i \tau_i$ with rollout inside C . We report the overhead ratio $C/\sum_i m_i \tau_i$ next to GPU hours, because a large ratio limits what Cost-GPAS can save.

5 RESULTS

5.1 NOISE AND ALLOCATION DYNAMICS

[TODO: Insert Figure 2 from the four-teacher logs: (a) scaled and raw noise by task over training; (b) GPAS and Cost-GPAS counts; (c) variance ratio H with the bound 2; (d) per-task time τ_i and the overhead ratio.]

Figure 2: Optimizer-scaled noise and the resulting allocations. [TODO: State the main quantitative observation in the caption.]

Table 1: Held-out gradient variance relative to Uniform at three Uniform checkpoints, for the allocation each rule computes from the other half of the fresh micro-batches. [TODO: Fill from the checkpoint statistics; add the relative error of $\hat{e}_i$ against the held-out half.]

Allocation	Early	Middle	Late
Uniform	1.00	1.00	1.00
RawNoise			
LossGap			
GPAS			
Cost-GPAS			

[TODO: Report the cross-task range and drift of $\hat{e}_i$, the fraction of steps in which raw and scaled noise rank the tasks differently, how often each task sits at a count bound, and the median and quartiles of H .]

5.2 TRAINING EFFICIENCY

[TODO: Insert Figure 3: (a) held-out weighted teacher loss versus optimizer step for Uniform, GPAS, RawNoise, and LossGap, with paired bootstrap bands over held-out prompts; (b) held-out weighted teacher loss versus GPU hours for Uniform, GPAS, and Cost-GPAS. Per-task curves go to the appendix.]

Figure 3: Held-out teacher loss at matched steps and at matched compute. [TODO: State the matched-step loss difference and the GPU-hour saving at Uniform’s final loss.]

5.3 CAPABILITY AND COMPUTE

Table 2: Four-teacher capability and efficiency. StdMOPD optimizes a different objective, so its held-out F is not comparable and its compute column is omitted. **[TODO: Populate from the evaluation outputs and timing logs; state every task-level difference from Uniform next to the average.]**

	Math	Code	IF	Science	Norm. avg.	Final held-out F	GPU h to target
Initial student							
Teachers							
StdMOPD							
Uniform							
RawNoise							
LossGap							
GPAS							
Cost-GPAS							

6 RELATED WORK

Task weighting and data allocation. Multi-task methods tune loss weights, alter gradient directions, or adapt data schedules (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018), and language-model training adapts domain, rollout, or prompt distributions from learning progress (Albalak et al., 2023; Wang et al., 2025; Zheng et al., 2025; Gao et al., 2025). These methods change the objective or the data distribution. GPAS keeps both fixed and changes only the number of micro-batches used to estimate each task gradient; because every task appears in every step, it observes within-task gradient noise for all tasks rather than for a sampled subset.

Multi-teacher on-policy distillation. MOPD routes student rollouts to task teachers and trains a shared student with token-level teacher feedback (Ma et al., 2026). Open-MOPD studies unequal token budgets, reward scales, and stale rewards (Gao et al., 2026); its token-share balancing holds here by construction through per-response averaging and w_i/m_i scaling. D³-MOPD adapts the domain mixture from the teacher-loss gap and its descent velocity (Sun et al., 2026), which changes the objective weights; GPAS changes the precision of each task-gradient estimate instead, and the two can be combined. Teacher mixtures can still produce transfer and task trade-offs (Li et al., 2026), which is why we report capability per task.

Stratified allocation for optimization. Gradient-norm sampling reduces the second moment of an SGD estimate (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Neyman allocation assigns samples in proportion to within-stratum standard deviation, and unequal sampling costs give the square-root cost correction used here (Neyman, 1934; Cochran, 1977; Mohri et al., 2026). The right variance scale depends on the optimizer: Euclidean coordinates suit SGD, whereas adaptive optimizers rescale coordinates (Lahire, 2023). GPAS applies Neyman allocation to fixed-size MOPD micro-batches with noise measured under the current AdamW scaling.

7 LIMITATIONS AND CONCLUSION

The study covers one student scale, four tasks, one seed per method, and one system layout. GPAS is a one-step rule: it treats micro-batches within a step as independent, holds the AdamW scaling fixed within the step, and its variance reduction is bounded by the count limits ($H \leq 2$). The additive step-time model was fitted to one layout, in which rollout dominates the fixed time; other layouts need their own fit. The method needs at least two micro-batches per task per step, so settings with many more tasks than micro-batches need a task-subsampling variant.

GPAS fixes the objective weights and adapts only how many micro-batches each task contributes, so the allocation changes gradient variance and nothing else. The counts follow Neyman allocation in AdamW coordinates, Cost-GPAS adds measured time, and both use statistics that MOPD training already produces. **[TODO: Replace this sentence with the main four-teacher empirical conclusion.]**

AI USE STATEMENT

We used generative AI tools for language editing and manuscript organization. The authors reviewed every assisted revision and take responsibility for the text, claims, proofs, code, and experimental results.

ETHICS STATEMENT

The study evaluates optimization methods for existing language models with reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation.

REPRODUCIBILITY STATEMENT

Section 4 specifies the micro-batch composition, fixed objective weights, integer allocation, resource accounting, baselines, held-out evaluation, and matched prompt streams. The released artifact includes controlled-check code and seeds, the run configuration file, model and tokenizer revisions, initial-loss weights, prompt streams, optimizer and allocation states, micro-batch counts, noise and timing logs, per-prompt evaluation output, and source data for every figure and table.

REFERENCES

- Guillaume Alain, Alex Lamb, Chinnadhurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL <https://arxiv.org/abs/1511.06481>.
- Alon Albalak, Liangming Pan, Colin Raffel, and William Yang Wang. Efficient online data mixing for language model pre-training, 2023. URL <https://arxiv.org/abs/2312.02406>.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL <https://proceedings.mlr.press/v80/chen18a.html>.
- William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.
- Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098>.
- Zhaolin Gao, Joongwon Kim, Wen Sun, Thorsten Joachims, Sid Wang, Richard Yuanzhe Pang, and Liang Tan. Prompt curriculum learning for efficient LLM post-training, 2025. URL <https://arxiv.org/abs/2510.01135>.
- Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL <https://proceedings.mlr.press/v80/katharopoulos18a.html>.
- Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL <https://opt-ml.org/papers/2023/paper20.pdf>.
- Zhaoyi Li, Deyang Kong, Yuan Wei, Evan Yang, Ranran Shen, Mahardika Krisna Ihsani, Ming Yang, Wei Zhang, Chuan Hao, Jian Yang, Ran Tao, Bryan Dai, Shikun Zhang, Wei Ye, Ying Wei, and Defu Lian. Every coin has two sides: On the dual nature of generalization in on-policy distillation of large language models, 2026. URL <https://arxiv.org/abs/2608.16647>.

- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL <https://arxiv.org/abs/2604.28020>.
- Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL <https://arxiv.org/abs/1810.04650>.
- Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D³-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL <https://arxiv.org/abs/2608.24987>.
- Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. DUMP: Automated distribution-level curriculum learning for RL-based LLM post-training, 2025. URL <https://arxiv.org/abs/2504.09710>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.
- Haizhong Zheng, Yang Zhou, Brian R. Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for LLM reasoning via selective rollouts, 2025. URL <https://arxiv.org/abs/2506.02177>.

A SUPPLEMENTARY TABLES, DERIVATIONS, AND DETAILS

A.1 NOTATION AND CONFIGURATION

Table A1: Notation and the values used in the four-teacher study.

Symbol	Meaning	Value in this study
M	number of tasks and teachers	4
w_i	fixed objective weight	inverse initial loss, normalized
G	micro-batches per optimizer step	16
b	prompts per micro-batch, one response each	4
m_i	task- i micro-batch count	$2 \leq m_i \leq 8$
$g_{i,s}$	unweighted micro-batch gradient	from the backward pass
D	AdamW diagonal scaling	$(\sqrt{v} + \epsilon)^{-1}$
$e_i, \hat{e}_i$	scaled one-micro-batch noise; its within-step estimate	estimated each step
τ_i, C	time per task- i micro-batch; fixed time per step	exponential averages
H	predicted variance ratio relative to Uniform	$H \leq 2$

Table A2: Four-teacher MOPD configuration.

Component	Configuration
Student	Qwen3-1.7B (Qwen Team, 2025) in non-thinking mode; [TODO: exact check-point revision and initial scores]
Teachers	Four Qwen3-1.7B models obtained from the student by reinforcement learning on each task (math, code, instruction following, science), so every teacher shares the student’s origin and tokenizer; non-thinking mode; [TODO: RL recipe, steps, revisions, and teacher scores]
Training prompts	16,000 unique prompts per task, shuffled once and read without replacement; [TODO: dataset names, versions, licenses, and filters]
Rollout and loss	One response per prompt; one update per rollout batch; sampled-token reverse KL, averaged over valid tokens and then prompts; maximum response length 4,096, truncated responses kept; [TODO: temperature, ratio-truncation rule, and observed truncation rate by task]
Step and budget	$b = 4$ prompts per micro-batch, $G = 16$ micro-batches per step, 500 steps, and 32,000 attempted responses; $2 \leq m_i \leq 8$
Optimization	AdamW; inverse-initial-loss objective weights fixed before training; [TODO: learning rate and schedule, β_1, β_2, ϵ, weight decay, gradient clipping, precision, and sharding]
Evaluation	Held-out teacher loss on 128 fixed prompts per task from checkpoints every 50 steps; MATH-500, LiveCodeBench slice, IFBench, GPQA-Diamond; [TODO: benchmark versions and decoding settings]
Seeds	1 per method
System	Two GPUs per run: one 96 GB GPU trains the student, one 48 GB GPU serves rollouts and holds the four resident teachers; [TODO: GPU models, inference engine, software revisions, and GPU hours per run]

A.2 ONE-STEP VARIANCE CRITERION

Let D be fixed within the step. If F is L -smooth and $\theta' = \theta - \eta DA$, then

$$\mathbb{E}[F(\theta')] \leq F(\theta) - \eta \nabla F^\top D \nabla F + \frac{\eta^2 L}{2} (\|D \nabla F\|_2^2 + V(m)). \quad (8)$$

For a fixed student state, D , and step size, only $V(m)$ depends on the allocation. This local bound motivates the variance criterion; the end-to-end experiment includes momentum and gradient clipping.

A.3 GRADIENT VARIANCE, ITS ESTIMATE, AND THE ALLOCATION RULES

Independence across tasks and micro-batches gives

$$\mathbb{E}\|D(A - \mathbb{E}[A])\|_2^2 = \sum_i w_i^2 \mathbb{E}\|D(\bar{g}_i - \mathbb{E}[g_i])\|_2^2 = \sum_i \frac{w_i^2}{m_i} \mathbb{E}\|D(g_i - \mathbb{E}[g_i])\|_2^2, \quad (9)$$

which is Equation 3. Within a step, $\sum_s \|D(g_{i,s} - \bar{g}_i)\|_2^2 = \sum_s \|Dg_{i,s}\|_2^2 - m_i \|D\bar{g}_i\|_2^2$, so Equation 6 is the usual unbiased sample variance summed over the scaled coordinates and is defined whenever $m_i \geq 2$.

Write $q_i = w_i \sqrt{e_i}$. With equal micro-batch cost, the Cauchy–Schwarz inequality

$$\left(\sum_i m_i\right) \left(\sum_i \frac{q_i^2}{m_i}\right) \geq \left(\sum_i q_i\right)^2 \quad (10)$$

holds with equality at $m_i \propto q_i$, which is GPAS. With task-dependent cost and zero fixed overhead,

$$\left(\sum_i m_i \tau_i\right) \left(\sum_i \frac{q_i^2}{m_i}\right) \geq \left(\sum_i q_i \sqrt{\tau_i}\right)^2, \quad (11)$$

with equality at $m_i \propto q_i / \sqrt{\tau_i}$. For $C > 0$, Cost-GPAS evaluates Equation 5 over the feasible integer counts. In both rules the continuous counts are clipped to $[m_{\min}, m_{\max}]$ by solving for the scale at which the clipped counts sum to G , and the result is rounded by largest remainder.

A.4 ALLOCATION AND ADAMW’S SECOND MOMENT

For a fixed allocation, assume task- i micro-batch gradients are independent with mean h_i and coordinatewise variance σ_i^2 . Equation 2 gives

$$\mathbb{E}[A] = \sum_i w_i h_i, \quad \mathbb{E}[A^2] = \left(\sum_i w_i h_i \right)^2 + \sum_i \frac{w_i^2}{m_i} \sigma_i^2, \quad (12)$$

so the noise term of AdamW’s second-moment observation depends on the allocation, as it does under any change of batch composition. Within the count bounds each task’s term lies between $0.5\times$ and $2\times$ its Uniform value, and the observation enters the update through the square root of an exponential average, so the update magnitude in a noise-dominated coordinate changes by less than a factor of $\sqrt{2}$ in the worst case. The experiments use conventional AdamW.

An allocation-independent observation is available if desired:

$$U = \sum_{i=1}^M \frac{w_i}{m_i} \sum_{s=1}^{m_i} g_{i,s}^2, \quad \mathbb{E}[U/G] = \frac{1}{G} \sum_i w_i (h_i^2 + \sigma_i^2), \quad (13)$$

whose expectation does not depend on m . At proportional allocation $m_i = Gw_i$ its noise term equals that of Equation 12; using U without the $1/G$ factor would enlarge the target by about G and shrink updates by about $\sqrt{G}$ in noise-dominated coordinates. In the controlled check below, reallocating from Uniform to GPAS changes the conventional A^2 observation by 0.543 in relative norm (calculated 0.542) and the U/G observation by 0.0009 (calculated 0).

A.5 CONTROLLED CALCULATION

The controlled check uses three Gaussian micro-batch gradients in three coordinates and a diagonal AdamW scaling that reverses their noise ranking. Every step contains $G = 16$ micro-batches with $2 \leq m_i \leq 12$. Bounded rounding gives counts $(6, 5, 5)$ for Uniform, $(11, 3, 2)$ for RawNoise, $(3, 3, 10)$ for GPAS, and $(3, 5, 8)$ for zero-overhead Cost-GPAS. All methods use the same fixed weights and common random numbers over 20,000 trials. GPAS attains $0.679\times$ Uniform’s scaled variance (calculated 0.681), RawNoise $2.316\times$, and Cost-GPAS a time-variance product of $0.857\times$ Uniform (calculated 0.854). The second-moment panel uses 200,000 simulated steps.

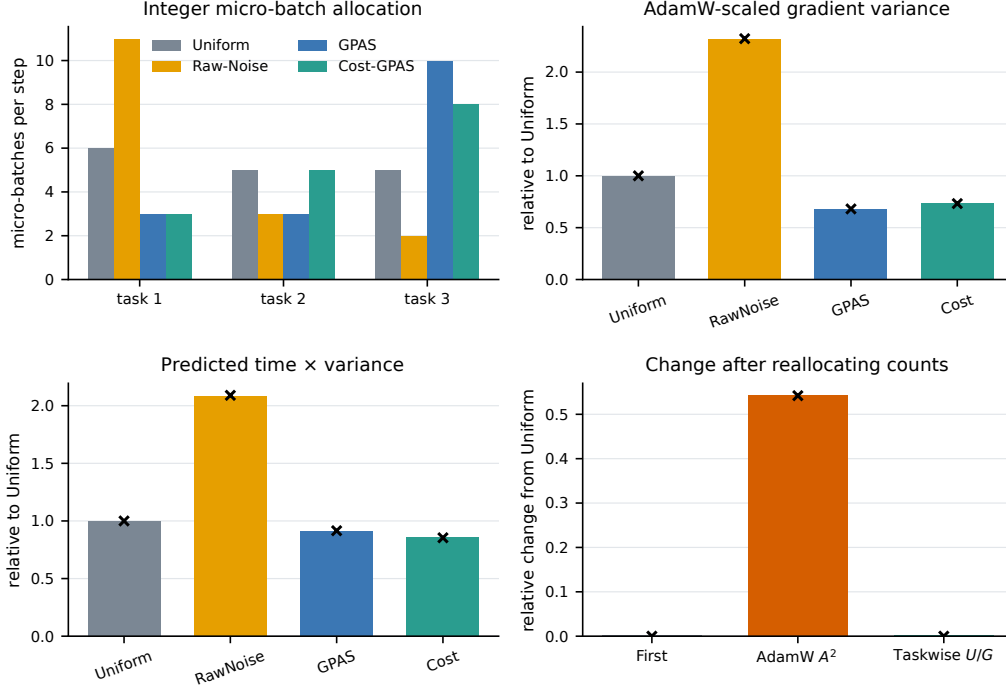


Figure 4: Controlled checks. Bars are Monte Carlo estimates and crosses are calculated values. GPAS uses AdamW-scaled within-task noise; Cost-GPAS also uses task cost. The last panel compares second-moment observations after an integer reallocation.

A.6 MOPD LOSS AND MICRO-BATCH NORMALIZATION

For a student-generated prefix z , write $q_v = \pi_\theta(v | z)$ and $t_v = T_i(v | z)$. The sampled-token reverse-KL score is

$$\text{sg} \left[\log \frac{q_V}{t_V} \right] \nabla_\theta \log q_V, \quad V \sim q. \quad (14)$$

The implementation averages valid tokens within one response and then the $b = 4$ responses from distinct prompts in a micro-batch. Any configured importance-ratio truncation is applied before these means; the fixed-objective factor w_i/m_i is applied after them. There is no groupwise advantage normalization and no outcome-reward term.

A.7 HELD-OUT GRADIENT VARIANCE PROTOCOL

At each of three Uniform checkpoints (steps **[TODO: 50, 250, 500]**) and for each task, we generate 32 fresh micro-batches from the held-out prompts and record $\|Dg_{i,s}\|_2^2$ and $\|D\bar{g}_i\|_2^2$ for each half, with D from the checkpoint’s AdamW state. Half $\mathcal{A}$ gives $\hat{e}_i^{\mathcal{A}}$, from which each rule computes its allocation $m^{\mathcal{A}}$ (LossGap uses the checkpoint’s held-out losses); half $\mathcal{B}$ gives $\hat{e}_i^{\mathcal{B}}$, and the held-out variance of that allocation is $\sum_i w_i^2 \hat{e}_i^{\mathcal{B}} / m_i^{\mathcal{A}}$, reported relative to the same quantity at Uniform counts. We exchange the halves and average. The estimator error $|\hat{e}_i^{\mathcal{A}} - \hat{e}_i^{\mathcal{B}}| / \hat{e}_i^{\mathcal{B}}$ is also computed from subsets of two and four micro-batches to match the online estimate. Only scalar norms are stored.

A.8 SYSTEM TIMING AND LOGS

For each task in a step, the log records rollout, scoring, backward, and optimizer time, peak memory, attempted responses, and valid and generated tokens. τ_i is the scoring and backward time of task i ’s micro-batches divided by m_i ; C is the remaining step time, including rollout, weight synchronization to the inference GPU, and the optimizer update. GPU hours are wall time times the two reserved

GPUs, including all waiting. Checkpoints store the allocation, noise estimates, timing averages, and prompt positions, and a resume test reproduces the next allocation and parameter update. A layout that scores tasks concurrently with training has a step time closer to $C + \max_i m_i \tau_i$ than to the serial sum; the fixed-objective scaling and GPAS are unchanged, and Cost-GPAS should use the step-time model fitted for that layout.

A.9 FULL TASK-LEVEL TRAJECTORIES

[TODO: Insert per-task held-out and training losses, prompt consumption, response length and truncation, count-bound occupancy, and step-time quantiles from the released logs.]

Figure 5: Task-level training and system traces. **[TODO: Replace after the four-teacher runs finish.]**